



/-

A5O-KS-2026-09-06 — Kill-switch survival under compromise
 Standalone model, core Lean 4, no Mathlib, no axioms beyond Classical.
 This file is NOT the AgentOBO/a5o-lean source; it is a self-contained
 formalisation of the claim made in the public post
 "Part 10 — Can a Compromised Agent Defeat Its Own Kill Switch?"
 (R. Shankar NRK, LinkedIn, w/c 2026-08-30).

-/

namespace A5O.KillSwitch

/-- A control is characterised by the trust roots it depends on
 (identity provider, CA, hardware key, a person-as-signer, an appointer). -/
 structure Control (Root : Type) where
 deps : Root → Prop

/-- Adversary state: the roots the adversary controls. -/
 def Compromise (Root : Type) := Root → Prop

variable {Root : Type}

/-- Forgeable: every root the control depends on is adversary-controlled. -/
 def Forgeable (c : Control Root) (C : Compromise Root) : Prop :=
 ∀ r, c.deps r → C r

/-- Survives: at least one root the control depends on is outside the adversary. -/
 def Survives (c : Control Root) (C : Compromise Root) : Prop :=
 ∃ r, c.deps r ∧ ¬ C r

/-- Full compromise of the agent: everything it depends on is adversary-controlled. -/
 def FullCompromise (a : Control Root) (C : Compromise Root) : Prop :=
 ∀ r, a.deps r → C r

/-- "Same well": the kill switch draws only on roots the agent also draws on. -/
 def SameWell (k a : Control Root) : Prop :=
 ∀ r, k.deps r → a.deps r

/-- Two signatures both required (the post's "hardware authenticator + second person").
 The adversary must forge both, so it depends on the union of roots. -/
 def Conj (k₁ k₂ : Control Root) : Control Root :=
 ⟨fun r => k₁.deps r ∨ k₂.deps r⟩

/-- Survival and forgeability are complementary. -/
 theorem survives_iff_not_forgeable (c : Control Root) (C : Compromise Root) :

```

    Survives c C  $\leftrightarrow$   $\neg$  Forgeable c C := by
  constructor
  · rintro ⟨r, hr, hnC⟩ hF
    exact hnC (hF r hr)
  · intro hnF
    exact Classical.byContradiction fun hns =>
      hnF fun r hr => Classical.byContradiction fun hnC => hns ⟨r, hr, hnC⟩

/-- T1 — the post's claim, verbatim in structure:
  a kill switch drinking from the same well as the agent is forged
  by any adversary that has fully compromised the agent. -/
theorem same_well_defeated
  (k a : Control Root) (C : Compromise Root)
  (hsw : SameWell k a) (hfc : FullCompromise a C) :
  Forgeable k C := by
  intro r hk
  exact hfc r (hsw r hk)

/-- T2 — a second signature from the same well adds nothing.
  Counting signers is not the variable. -/
theorem second_signature_same_well_defeated
  (k₁ k₂ a : Control Root) (C : Compromise Root)
  (h₁ : SameWell k₁ a) (h₂ : SameWell k₂ a) (hfc : FullCompromise a C) :
  Forgeable (Conj k₁ k₂) C := by
  intro r hr
  cases hr with
  | inl h => exact hfc r (h₁ r h)
  | inr h => exact hfc r (h₂ r h)

/-- T3 — necessity: any kill switch that survives full compromise of the agent
  depends on a root the agent does not. Independence is not optional. -/
theorem survival_requires_independent_root
  (k a : Control Root) (C : Compromise Root)
  (hs : Survives k C) (hfc : FullCompromise a C) :
   $\exists$  r, k.deps r  $\wedge$   $\neg$  a.deps r := by
  obtain ⟨r, hk, hnC⟩ := hs
  exact ⟨r, hk, fun ha => hnC (hfc r ha)⟩

/-- T4 — the boundary. Against the minimal full compromise (C = exactly the
  agent's roots), survival is EQUIVALENT to not sharing the well.
  This is "know exactly where independence ends", as a biconditional. -/
theorem boundary (k a : Control Root) :
  Survives k a.deps  $\leftrightarrow$   $\neg$  SameWell k a := by
  constructor

```

```

· rintro ⟨r, hk, hna⟩ hsw
  exact hna (hsw r hk)
· intro hns
  exact Classical.byContradiction fun hns =>
    hns fun r hk => Classical.byContradiction fun hna => hns ⟨r, hk, hna⟩

/-- T5 — independence is necessary, not sufficient. An independent root that
the adversary ALSO holds gives no survival. The post's remedy is a design
choice; survival is a property of the compromise set, not of the design. -/
theorem independent_root_not_sufficient
  (k a : Control Root) (C : Compromise Root)
  (_hfc : FullCompromise a C)
  (_hind : ∃ r, k.deps r ∧ ¬ a.deps r)
  (hCk : ∀ r, k.deps r → C r) :
  ¬ Survives k C := by
  rintro ⟨r, hk, hnC⟩
  exact hnC (hCk r hk)

/-- A5O mapping. An appointment names the appointer's root, and the agent's
authority is DERIVED from the appointment — so the appointer's root is
upstream of, and not among, the agent's dependencies. -/
structure Appointment (Root : Type) where
  appointer : Root
  agent      : Control Root
  upstream   : ¬ agent.deps appointer

/-- Revocation addressed to the appointer. -/
def Revocation (A : Appointment Root) : Control Root :=
  ⟨fun r => r = A.appointer⟩

/-- T6 — revocation dominance under the minimal full compromise:
a kill switch keyed to the appointer survives total compromise of
everything the agent depends on, by construction, with no second person. -/
theorem revocation_survives_minimal_compromise (A : Appointment Root) :
  Survives (Revocation A) A.agent.deps :=
  ⟨A.appointer, rfl, A.upstream⟩

/-- T7 — and against any compromise set at all, revocation survives exactly
when the appointer's root is outside it. The post's question
"under what conditions do they stop being real?" has one answer:
when the appointer is compromised — and nothing else. -/
theorem revocation_survives_iff (A : Appointment Root) (C : Compromise Root) :
  Survives (Revocation A) C ↔ ¬ C A.appointer := by
  constructor

```

```
· rintro ⟨r, hr, hnC⟩  
  cases hr  
  exact hnC  
· intro h  
  exact ⟨A.appointer, rfl, h⟩
```

```
end A5O.KillSwitch
```